ESCOLA DE NEGOCIOS, TECNOLOGIA E INOVACAO DO CESUPA
Teoria da Computabilidade | Prof. Daniel Leal Souza | 01/2026
Turmas CC5MA / CC5NA

PROGRAMAS, MAQUINAS E EQUIVALENCIA

Trabalho de Implementação AV1

Função Computada

Potenciação Inteira: f(base, exp) = base^{exp}

Programa Monolítico	Programa Iterativo
Programa Recursivo	Máquina de Traços

EQUIPE

Andrey Lourival Andrade Garcia

Filipe Cesar Maciel Sucupira

Everton Gustavo de Oliveira

Igor Cecim Vilhena

Belém, Pará — Abril de 2026

SEÇÃO 1 Identificação da Equipe

Integrante	Curso
Andrey Lourival Andrade Garcia	Ciência da Computação – 5º período
Filipe César Maciel Sucupira	Ciência da Computação – 5º período
Everton Gustavo de Oliveira	Ciência da Computação – 5º período
Igor Cecim Vilhena	Ciência da Computação – 5º período

Disciplina	Teoria da Computabilidade
Professor	Daniel Leal Souza
Semestre	01/2026 | Turmas CC5MA
Linguagens	C (monolítico) • Python (iterativo e recursivo) • Python (Máquina de Traços)

Repositório	https://github.com/IgVilhena/Atividade-Pr-tica-AV1-Teoria-da-Computa-o-.git

SEÇÃO 2 Definição da Função Computada

A função escolhida é a **Potenciação Inteira**: dados um inteiro *base* e um inteiro não-negativo *exp*, a função retorna o produto de *base* por si mesmo *exp* vezes. Formalmente: **f(base, exp) = base^{exp}**.

Justificativa da escolha. A potenciação inteira foi escolhida por apresentar uma estrutura recursiva natural ($base^{exp} = base \times base^{exp-1}$), que permite implementação direta nos três estilos estudados sem ambiguidade. O caso base (exp = 0) encerra a recursão de forma clara, e a versão monolítica normalizada (L1–L7) é compacta o suficiente para que a Máquina de Traços produza cadeias legíveis em sala de aula.

Especificação formal

Domínio	base ∈ Z, exp ∈ N (exp ≥ 0)
Contradomínio	Z
Caso base	f(base, 0) = 1 (qualquer número elevado a 0 é 1)
Passo	f(base, exp) = base × f(base, exp−1)
Operação	Multiplicação iterada: result = 1 × base × base × ... × base (exp vezes)

Exemplos de entrada e saída

Entrada (base, exp)	Saída	Verificação
(2, 0)	1	caso base
(2, 1)	2	$2^1 = 2$
(2, 3)	8	2×2×2 = 8
(3, 4)	81	3×3×3×3 = 81
(5, 3)	125	5×5×5 = 125
(10, 2)	100	10×10 = 100

SEÇÃO 3 Descrição da Modelagem Adotada

A função f(base, exp) foi implementada em três estilos de programação distintos, cada um utilizando apenas os primitivos próprios daquele paradigma. O programa monolítico foi normalizado em instruções atômicas rotuladas (L1–L7), compatíveis com a formalização exigida pela Máquina de Traços.

Estilo	Linguagem	Primitivo central	Variáveis de controle
Monolítico	C	goto / rótulos L1–L7	result, count (+ base, exp)
Iterativo	Python	while count < exp	result, count

Recursivo	Python	chamada recursiva + caso base	pilha de ativação implícita

SEÇÃO 4–5

Implementação e Explicação dos Três Programas

4.1 Programa Monolítico (C com goto)

O programa monolítico é caracterizado pelo fluxo de controle explícito via **goto** e rótulos. Não há estruturas de repetição de alto nível (*for/while*) nem chamadas de função — apenas desvios condicionais e incondicionais entre rótulos. Cada instrução é atômica e corresponde exatamente a um passo da Máquina de Traços.

```
/* Instruções normalizadas — forma compatível com a Máquina de Traços */ L1:
result = 1;
L2: count = 0;
L3: if (count >= exp) goto L7; /* teste de parada */
L4: result = result * base; /* multiplicação acumulada
*/ L5: count = count + 1; /* incremento do contador */
L6: goto L3; /* retorno ao teste */ L7: /* HALT */
printf("%d^%d = %d", base, exp, result);
```

Característica monolítica: todo o controle é feito por goto/rótulos. Não existe while, for, nem recursão. O loop é formado pelo par L3 → L6 → L3.

4.2 Programa Iterativo (Python com while)

O programa iterativo utiliza a estrutura de repetição **while** de forma explícita. A variável *count* cresce de 0 até *exp*, controlando o número de multiplicações acumuladas em *result*.

```
def potencia_iterativa(base: int, exp: int) -> int:
result = 1 # acumulador (elemento neutro da multiplicação)
count = 0 # contador de iterações while count < exp: #
estrutura de repetição explícita result = result * base #
operação central count = count + 1 # avança o contador
return result # resultado após exp multiplicações
```

Característica iterativa: o loop **while** é a única estrutura de repetição. A condição de parada (count < exp) é testada no início de cada iteração.

4.3 Programa Recursivo (Python com chamada recursiva)

O programa recursivo resolve o problema pela redução sucessiva: $base^{exp} = base \times base^{exp-1}$. A condiçãobase encerra a recursão quando exp = 0, retornando 1. Não há loop — o controle é dado pela pilha de ativação.

```
def potencia_recursiva(base: int, exp: int) -> int: # ■
CONDIÇÃO-BASE
■■■
```

SEÇÃO 6 Exemplos de Execução e Resultados

Os três programas foram executados para as mesmas entradas, confirmando que todos computam a mesma função:

```
$ python3 iterativo.py
potencia_iterativa( 2, 0) = 1
potencia_iterativa( 2, 1) = 2
potencia_iterativa( 2, 3) = 8
potencia_iterativa( 3, 4) = 81
potencia_iterativa( 5, 3) = 125
potencia_iterativa(10, 2) = 100
```

```
$ python3 recursivo.py potencia_recursiva( 2, 3) = 8 ---
Rastreamento de chamadas para 2^3 --potencia_recursiva(2, 3)
potencia_recursiva(2, 2) potencia_recursiva(2, 1)
potencia_recursiva(2, 0) => caso base: retorna 1
=> 2 * 1 = 2
=> 2 * 2 = 4
=> 2 * 4 = 8
$ gcc monolitico.c -o monolitico $ echo -e
"2\n3" | ./monolitico
Resultado: 2^3 = 8
```

Nota de execução: o programa monolítico em C usa dois *scanf* separados; para alimentá-lo via pipe é necessário separar os valores com nova linha: `echo -e "2\n3" | ./monolitico`. O comando `echo '2 3'` passaria ambos na mesma linha e não funcionaria corretamente.

Todos os três programas retornam o mesmo valor para todas as entradas testadas. Isso demonstra que computam a mesma **função computada** f(base, exp) = $base^{exp}$.

SEÇÃO 7 Aplicação da Máquina de Traços

A Máquina de Traços (MT) executa um programa monolítico normalizado passo a passo e registra cada configuração como um elemento da cadeia de traços. Cada configuração σ é o par (rótulo atual, estado das variáveis). A cadeia completa é: $\sigma_0 \blacksquare \sigma_1 \blacksquare \sigma_2 \blacksquare \ldots \blacksquare \sigma_n$.

Programas analisados pela MT (entrada: base=2, exp=3)

Rótulo	P1 — contagem crescente	P3 — contagem decrescente
L1	result 1	result 1
L2	count 0	count exp DIFERE
L3	SE count ≥ exp → L7	SE count ≤ 0 → L7 DIFERE
L4	result result × base	result result × base
L5	count count + 1	count count − 1 DIFERE
L6	GOTO L3	GOTO L3
L7	HALT	HALT

Células destacadas em vermelho indicam as instruções que diferem entre P1 e P3.

Cadeia de traços de P1 — base=2, exp=3

σ	Rótulo	result	count	Operação
σ_0	L1	—	—	início
σ_1	L2	1	—	result ← 1
σ_2	L3	1	0	count ← 0
σ_3	L4	1	0	0 < 3 → continua
σ_4	L5	2	0	result ← 1×2 = 2
σ_5	L6	2	1	count ← 1
σ_6	L3	2	1	GOTO L3
σ_7	L4	2	1	1 < 3 → continua
σ_8	L5	4	1	result ← 2×2 = 4
σ_9	L6	4	2	count ← 2
σ_{10}	L3	4	2	GOTO L3
σ_{11}	L4	4	2	2 < 3 → continua
σ_{12}	L5	8	2	result ← 4×2 = 8
σ_{13}	L6	8	3	count ← 3
σ_{14}	L3	8	3	GOTO L3
σ_{15}	L7	8	3	3 ≥ 3 → HALT ✓

Formato de fita — P1 (base=2, exp=3)

O formato de sequência de fita representa a cadeia como uma série de configurações (rótulo, estado) encadeadas pelo símbolo ■ ("resulta em"):

```
(L1,{})  ■
(L2,{result=1})  ■
(L3,{result=1, count=0})  ■
(L4,{result=1, count=0})  ■
(L5,{result=2, count=0})  ■
```

```
(L6,{result=2, count=1})
```

```
(L3,{result=2, count=1})
(L4,{result=2, count=1})
(L5,{result=4, count=1})
(L6,{result=4, count=2})
(L3,{result=4, count=2})
(L4,{result=4, count=2})
(L5,{result=8, count=2})
(L6,{result=8, count=3})
(L3,{result=8, count=3})
(L7,{result=8, count=3})  HALT
```

Cadeia de traços de P3 — base=2, exp=3

σ	Rótulo	result	count	Operação
σ_0	L1	—	—	início
σ_1	L2	1	—	result 1
σ_2	L3	1	3	count exp = 3 DIVERGE
σ_3	L4	1	3	3 > 0 → continua
σ_4	L5	2	3	result 1×2 = 2
σ_5	L6	2	2	count 2
σ_6	L3	2	2	GOTO L3
σ_7	L4	2	2	2 > 0 → continua
σ_8	L5	4	2	result 2×2 = 4
σ_9	L6	4	1	count 1
σ_{10}	L3	4	1	GOTO L3
σ_{11}	L4	4	1	1 > 0 → continua
σ_{12}	L5	8	1	result 4×2 = 8
σ_{13}	L6	8	0	count 0

σ_{14}	L3	8	0	GOTO L3
σ_{15}	L7	8	0	0 ≤ 0 → HALT ✓

Linha destacada em vermelho (σ_2) indica o ponto de divergência: após L2, P1 tem count=0 enquanto P3 tem count=3.

SEÇÃO 8 Demonstração de Equivalência Forte — P1 ≡ Iterativo Normalizado

✓ P1 ≡ Iterativo Normalizado — FORTEMENTE EQUIVALENTES

O programa iterativo (Python), quando transcrito para a forma monolítica normalizada exigida pela MT, produz exatamente as mesmas instruções L1–L7 de P1 na mesma ordem. Dois programas são **fortemente equivalentes** se e somente se, para toda entrada, suas cadeias de traços são idênticas configuração a configuração — não basta que produzam o mesmo resultado final.

Forma normalizada do programa iterativo (Python → L1–L7):

```
# Python iterativo # Forma normalizada (L1-
L7) result = 1 → L1: result  1 count = 0 → L2:
count  0
while count < exp: → L3: SE count ≥ exp ENTÃO GOTO L7
result = result * base → L4: result  result × base
count = count + 1 → L5: count  count + 1
# (volta ao while) → L6: GOTO L3
# (fim do while) → L7: HALT
```

A condição do **while** Python é *count* < *exp*, equivalente ao desvio *SE count* ≥ *exp ENTÃO GOTO L7* de L3 — a negação da guarda. A cada iteração, as atribuições são feitas na mesma ordem (L4 → L5) e o retorno ao teste é implícito no loop Python, correspondendo a L6. Portanto, as instruções normalizadas são idênticas às de P1.

σ	P1 (L, estado)	Iterativo Normalizado (L, estado)	Iguais?
σ_0	L1 | —	L1 | —	✓
σ_1	L2 | result=1	L2 | result=1	✓
σ_2	L3 | count=0, result=1	L3 | count=0, result=1	✓
σ_3	L4 | count=0	L4 | count=0	✓
σ_4	L5 | result=2	L5 | result=2	✓
…	…	…	✓
σ_{15}	L7 | result=8, count=3	L7 | result=8, count=3	✓

Conclusão: as 16 configurações de P1 e do iterativo normalizado são idênticas para base=2, exp=3, e serão idênticas para qualquer entrada, pois as instruções normalizadas são as mesmas. Logo, **P1 ≡ Iterativo** (equivalência forte). Esta equivalência é não-trivial: demonstra que um programa de alto nível (*while*) e um programa monolítico com goto computam não apenas o mesmo resultado, mas seguem exatamente a mesma sequência de configurações quando normalizados.

SEÇÃO 9 Demonstração de Não-Equivalência Forte — P1 ╳ P3

╳ P1 ╳ P3 — NÃO FORTEMENTE EQUIVALENTES

Embora P1 e P3 produzam o mesmo resultado final (result=8) para base=2, exp=3, suas cadeias de traços divergem já em σ_2 — imediatamente após a instrução L2. A divergência está no **valor da variável** ***count*** e, por consequência, no **teste realizado em L3** e na **condição de parada**:

Classificação do ponto de divergência (terminologia formal):

Aspecto	P1	P3	Diverge?
Instrução L2 (atribuição de count)	count 0 (inicialização fixa)	count exp (inicialização com entrada)	╳ **SIM**
Instrução L3 (teste realizado)	SE count ≥ exp → L7 (guarda crescente)	SE count ≤ 0 → L7 (guarda decrescente)	╳ **SIM**
Instrução L5 (operação sobre count)	count count + 1 (incremento)	count count − 1 (decremento)	╳ **SIM**
Condição de parada atingida quando	count = exp (count chega em exp)	count = 0 (count chega em 0)	╳ **SIM**
Resultado final (result)	8	8	✓ **IGUAL**

σ	P1 — estado em L3	P3 — estado em L3	Iguais?
σ_2	count=0 (inicializado com 0)	count=3 (inicializado com exp)	╳
σ_6	count=1 (crescente)	count=2 (decrescente)	╳
σ_{10}	count=2 (crescente)	count=1 (decrescente)	╳
σ_{14}	count=3 (crescente)	count=0 (decrescente)	╳
σ_{15}	HALT | count=3	HALT | count=0	╳

Ponto de divergência: σ_2 — após executar L2, P1 tem count=0 enquanto P3 tem count=3. A divergência ocorre na **instrução L2** (atribuição diferente), propaga-se para o **teste realizado em L3** (guarda invertida)

e para a **condição de parada** (count chega em exp vs. chega em 0). Todos os estados seguintes diferem, pois count evolui em sentidos opostos.

Resultado final igual ≠ equivalência forte. A MT exige identidade de toda a cadeia de configurações, não apenas da saída. P1 e P3 são funcionalmente equivalentes (computam a mesma função), mas não são fortemente equivalentes.

Caso especial — exp=0: quando exp=0, P1 e P3 convergem para a mesma cadeia (ambos inicializam count=0 e entram em HALT imediatamente em σ_2), mas isso não é suficiente para equivalência forte — basta *uma* entrada em que as cadeias diferem (e.g., exp=3).

SEÇÃO 10 Conclusão

Este trabalho demonstrou, na prática, os três estilos de programação estudados na disciplina e a aplicação formal da Máquina de Traços para análise de equivalência forte. As principais conclusões são:

- A função potenciação inteira ($base^{exp}$) foi corretamente implementada nos três estilos, computando a mesma função para todas as entradas testadas.
- O programa monolítico explicita o controle de fluxo via goto/rótulos, sem abstrações de alto nível, refletindo fielmente a formalização da MT.
- O programa iterativo encapsula o loop no **while**, tornando o código mais legível; sua forma normalizada L1–L7 é idêntica à de P1, o que fundamenta a equivalência forte não-trivial.
- O programa recursivo expressa diretamente a definição matemática da função, usando a pilha de ativação como mecanismo de controle.
- A Máquina de Traços revelou que **P1 e o iterativo normalizado são fortemente equivalentes** (cadeias idênticas) e que **P1 e P3 não são**, divergindo em σ_2 — na instrução L2, no teste de L3 e na condição de parada — mesmo produzindo o mesmo resultado final.
- **Resultado final igual não implica equivalência forte.** A MT exige identidade de toda a sequência de configurações.

A análise pela Máquina de Traços é mais rigorosa do que a simples comparação de resultados: dois programas com comportamentos internos distintos podem coincidir na saída para algumas entradas, mas não são equivalentes segundo o critério formal da disciplina.

Referências

- DIVERIO, Tiaraju Asmuz; MENEZES, Paulo Fernando Blauth. *Teoria da Computação*. Porto Alegre: Sagra Luzzatto.
- Material de aula — Prof. Daniel Leal Souza — CESUPA 2026/01.
- Repositório da equipe: https://github.com/IgVilhena/Atividade-Pr-tica-AV1-Teoria-da-Computa-o-.git

- Ferramentas de IA utilizadas: **Claude (Anthropic)** — apoio à redação, revisão de código e estruturação do documento. Trechos aproveitados: sugestões de formatação das tabelas de traços e revisão ortográfica dos parágrafos descritivos. Toda a lógica, os programas, as cadeias de traços e as conclusões foram definidos, implementados e validados pela equipe; a IA não gerou nenhum código ou análise autônoma — apenas revisou e formatou conteúdo já elaborado pelos integrantes.